

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA INGENIERÍA
CARRERA DE INGENIERÍA EN SOFTWARE

INFORME DE EXPOSICIÓN DE INGENIERÍA DE REQUERIMIENTOS

Análisis comparativo de herramientas CASE, modelado UML, MDE y generación de código

Estudiante: Alvia Villegas Erick Adalberto

Asignatura: Ingeniería de Requerimientos

Tema:
Herramientas de Modelado y Generación Automática de Código

Lunes, 20 de Julio de 2026

1. Introducción

El presente informe documenta de manera detallada las exposiciones realizadas el día lunes sobre las distintas herramientas CASE y entornos de modelado aplicados a la Ingeniería de Requerimientos y al Desarrollo Dirigido por Modelos (MDE/MDD). A continuación se transcribe el contenido integro recopilado en los apuntes principales, complementado con la estructura detallada de cada grupo, sus integrantes, herramientas expuestas, ventajas, limitaciones y demostraciones prácticas realizadas.

2. Desarrollo y Transcripción por Grupos de Exposición

Grupo H: Enterprise Architect

Herramienta: Enterprise Architect

TRANSCRIPCIÓN DE APUNTES

Soporta muchos estándares como UML, BPMN, SysML y MDE.

Nota: Permite crear CU (Casos de Uso), clases, diagramas de estado, etc.

Pilares:

- Gestión y trazabilidad.
- Documentación e ingeniería.
- Simulación y colaboración.

Tipos / Ventajas de EA:

- Simple e integrado.
- Permite trabajar en equipos complejos.

Limitaciones:

- Curva de aprendizaje muy alta.

Práctico:

En la práctica, Vera muestra cómo crear un proyecto y crear clases y sus atributos, mientras Huacón creó una clase en diagrama completo, añadió atributos, etc., y al final generó el código.

Grupo A: StarUML

Expositores / Integrantes: Díaz, Escudero, Mero, Mero S.

TRANSCRIPCIÓN DE APUNTES

Exposiciones individuales:

- **Díaz:** Mencionó los fundamentos MDD. Se dirige por Modelo MDE, MDA, etc. Explica los fundamentos de MDD (inicio de diagrama a código antes de programar). Los modelos son la base del desarrollo. MDA organiza el desarrollo en CIM, PIM y PSM. MDD usa modelos M2M (Modelo a Modelo) y M2T (Modelo a Texto / Código).
- **Escudero:** Indicó que se escogió por ser una herramienta muy útil para el UML con una gran generación de código. Se utiliza por compatibilidad con UML 2.5, además de que el proyecto se basa en código C# y sincronización de código y diagrama viceversa. Usó un proyecto de gestión para un gimnasio con registro de clientes y membresías.
- **Mero:** Explicó el manejo de las clases y explicación de abstracción en una clase Persona. Usó un diagrama de Modelo UML, diagrama de caso de uso, y explicó relaciones, atributos, cardinalidades y funcionamiento. Comparativa de StarUML y Visual Paradigm.
- **Mero S.:** Habló sobre las limitaciones y beneficios, como su facilidad de uso y que es un sistema aún en desarrollo. Destaca ventajas como multiplataforma, muy ligero, código abierto, interfaces intuitivas y exportes flexibles. Como limitaciones: versiones gratuitas limitadas y poca compatibilidad entre versiones.

Práctico:

En la práctica se mostró la interfaz con advertencia de licencia, se abrió el proyecto de presentación y se explicó sobre las extensiones para exportar. Se detalló el guardado de clases en el repositorio y el uso de Visual Studio para mostrar y abrir el código generado. Se editó el diagrama para mostrar la sincronización con el código mediante el menú de herramientas C#. Se concluyó que tener conocimientos básicos en UML permite sacar el máximo provecho de esta herramienta versátil.

Grupo G: UmpleOnline

Expositores / Integrantes: Alvia, Arboleda, Calle

TRANSCRIPCIÓN DE APUNTES

Trata las construcciones UML como elementos de primer nivel en el lenguaje. Genera automáticamente código y diagramas en lenguajes como Java, C++, Python, SQL.

Modalidades y Características:

- Modo código.
- Modo gráfico.
- Cero redundancias.
- Modelado estático.
- Máquinas de Estado.
- Trazabilidad y OCL.

Ciclo de desarrollo:

Concepción → Sincronización → Simulación

Exposiciones individuales y Práctico:

- **Alvia:** Presentó la herramienta intuitiva y útil para modelado de diagramas. Explicó cómo simplifica el código y maneja múltiples lenguajes de manera sincrónica (código a diagrama y viceversa). Mostró la web de la herramienta y el proyecto de la camaronera en C# (en fase beta para facilitar el proceso).
- **Arboleda:** Explicó el modelado trabajando sincronizadamente con el código (Código → Modelado / Modelado → Código). En la práctica preparó el código para mostrar su respectivo diagrama.
- **Calle:** Detalló el ciclo de desarrollo donde funciona la herramienta y su facilidad de uso. En la práctica generó un diagrama de clases exportando en JAVA y mostrando los resultados. En lo práctico se mostró la dualidad de trabajo y exportación.

Grupo C: Visual Paradigm

Expositores / Integrantes: Castro, Mero, Vera, Villafuerte, Alex

TRANSCRIPCIÓN DE APUNTES

Desarrollo teorico práctico:

- **Castro:** Habló sobre MDE, MDA y MDD, donde cada uno tiene sus funciones. Fundamentos de modelado donde los modelos pasan de ser documentación a ser la fuente principal.
- **Vera:** Escogieron este programa porque es de uso AIO (All-In-One), te permite realizar diferentes tipos de trabajos UML con archivos nativos .vpp. Mencionó que no genera automáticamente código directamente sin configuración previa. Se usó el proyecto SIMPA.
- **Perfiles y Estereotipos:** Permite definir perfiles como *Entity*, *Service* y *Repository*, los cuales clasifican los elementos.
- **Alex:** Explicó la elección del lenguaje JAVA por compatibilidad con JDK 8 o superior y mapeo principal con asociaciones múltiples. Muestra cómo generar el código previa verificación.

Práctico:

En Visual Paradigm, Castro usó un diagrama ya hecho sobre un sembrio y explicó sobre la asociación y diagramación, mostrando cómo usar las propiedades y demás en el programa. Mientras Mero/Vera mostró cómo generar código C# y JAVA. Muestra la interfaz del repositorio principal, diagrama de casos de uso, flujo de clases, atributos, operaciones y relaciones. Explicó las limitaciones de generación en versión Freemium (en C# solo base estructural) y modificó clases añadiendo atributos para actualizar sincrónicamente el código.

Grupo B: Bouml

Expositores / Integrantes: Arteaga, Guerrero

TRANSCRIPCIÓN DE APUNTES

MDE: MDA = CIM, PIM, PSM (Modelo Independiente de Plataforma / Modelo Específico de Plataforma / Computación).

MDD: Metodología que usa modelos como artefactos principales. Genera código automáticamente a partir del modelo UML.

Esta herramienta trabaja con ingeniería directa y también tiene un proceso de configuración que genera código automáticamente. Permite Ingeniería Inversa (genera UML desde código existente) y Roundtrip (sincronización bidireccional).

Ventajas:

- Velocidad excepcional.
- Bajo consumo de recursos.
- Soporte constante y multiplataforma.
- Gratuito y de código abierto.
- Usa C++ como lenguaje nativo para procesamiento rápido.

Limitaciones:

- Curva de aprendizaje más amplia.

Práctico:

En la práctica se utilizó un proyecto ya elaborado (sistema de veterinaria con gestión de citas) mostrando cómo la herramienta utiliza ingeniería inversa para el desarrollo automático de diagramas y códigos. Se cargó el diagrama prediseñado con sus clases, atributos, cardinalidades y relaciones, generando código en C++ y sincronizando hacia Visual Studio.

Grupo D: Modelio

Expositores / Integrantes: Crespo y equipo

TRANSCRIPCIÓN DE APUNTES

Es una herramienta para modelar UML y BPMN que incorpora Java EE y modalidad SaaS.

Características principales:

- Modelado completo de UML, BPMN 2 y ArchiMate.
- Soporte multiestándar y arquitectura modular.
- Organización por capas, jerarquías e interfaz editable.
- Verificación de consistencia.
- Modelado de arquitectura.

Ventajas y Desventajas:

- **Ventajas:** Open Source y gratuito (Licencia GPLv3 desde 2011), comunidad activa, más de 20 años de trayectoria, multiplataforma, reduce tiempos de documentación.
- **Desventajas:** Curva de aprendizaje alta, funciones avanzadas de pago, requiere ajustes manuales en la generación de código para ciertos diagramas, no preserva código mutado perfectamente.

Práctico:

Muestra la interfaz principal, da especificaciones y prepara el entorno para el desarrollo de un diagrama de clases y su jerarquía de carpetas. Crespo explicó la generación de código en JAVA a partir del diagrama creado, almacenando artefactos y abriendo el resultado en bloc de notas. Finalmente, se presentó un diagrama de estados explicando su ejecución.

Grupo E: Eclipse Papyrus + Acceleo

Herramientas: Eclipse Papyrus, Acceleo

TRANSCRIPCIÓN DE APUNTES

Herramienta de modelado mediante un plugin para exportar el código. Explica el MDD partiendo del diseño, modelo, reglas, salida y código, además de contener otros modelos como MDE, MDA y MDD para ser más amplio, formal y operativo.

Modelos de abstracción como CIM (procesos y requisitos), PIM (punto de vista de diseño sin uso de lenguaje de salida), PSM (especificación de artefactos). Transformación de MDD y los pasos entre niveles a Código.

¿Qué es Papyrus y Acceleo?

Papyrus es una herramienta de UML basada en Eclipse, permite editar diagramas y diseño con compatibilidad con soporte de OMG, siendo software de uso libre y compatibilidad con plugins. Se implementa Acceleo al entorno de Eclipse y sus herramientas.

¿Por qué? Funcionan bien juntas, son de código abierto y ambas trabajan con los estándares OMG para generar código aunque no puede hacerse al revés (no inversa). Beneficios del MDD: tiene una automatización y trazabilidad para llevar un orden en caso de presentarse algún error por medio de sus plantillas. Conclusión: MDD alinea el diseño a través del modelo, elevando la calidad y el desarrollo de software.

Práctico:

Muestra el entorno Eclipse con Papyrus y el plugin de Acceleo, explica cómo preparar el entorno para realizar un diagrama con Acceleo, muestra la carpeta con sus artefactos y pega la plantilla creando una carpeta para almacenar los códigos a partir del diagrama en lenguaje JAVA. Selecciona una carpeta para guardar los recursos y procede a solucionar un error de la plantilla, ejecuta con Acceleo y exporta generando el código del diagrama junto a sus clases, atributos y operaciones. Realiza un cambio al diagrama (clase estanque) para añadir un nuevo atributo y actualizar/generar código para mostrar los cambios.

3. Conclusiones Generales

Las exposiciones permitieron evaluar de forma integral el ecosistema de herramientas CASE para la Ingeniería de Requerimientos y la metodología MDE/MDD. Se evidenció cómo herramientas como **UmpleOnline** y **Bouml** priorizan la agilidad y sincronización bidireccional inmediata, mientras que

plataformas de gran escala como **Enterprise Architect**, **Visual Paradigm** y **Eclipse Papyrus** ofrecen marcos robustos para la gestión de arquitecturas complejas a costa de una mayor curva de aprendizaje.